

Military Situational Awareness and Surveillance

Nutan Kumari

2022341

[mail](#)

Pandillapelly Harshvardhini

2022345

[mail](#)

Abstract

Effective and precise detection of military-related noises is required due to the increasing dependence of military activities on audio-based surveillance systems. The classification of audio data from a military dataset that includes noises like gunfire, vehicle noise, and aircraft activity is the main goal of this study. Mel-Frequency Cepstral Coefficients (MFCCs) extraction, audio signal preprocessing, and using spectrogram-based features to record the signals' time-frequency properties are important processes. To create a reliable categorization system, these processed features are used in machine learning and deep learning pipelines. High accuracy in recognizing important noises is the goal in order to provide a trustworthy instrument for military surveillance and threat identification.

1. Introduction

The increasing reliance on audio-based surveillance systems in military operations underscores the need for accurate and efficient sound classification. Military audio data presents unique challenges, as it often comprises a wide range of sounds, such as gunfire, vehicle engines, and aircraft noises. Detecting and classifying these sounds is a critical task in ensuring situational awareness and operational security.

Recognizing military sounds often requires analyzing intricate patterns in the audio spectrum, including frequency, amplitude, and time-based variations. Human analysts, while adept, can struggle to interpret these signals accurately in real-time without sophisticated tools. Recent advancements in machine learning and deep learning have demonstrated promise in addressing these challenges by enabling automated feature extraction and classification.

On the deep learning front, Convolutional Neural Networks (CNNs) are employed for extracting local patterns from spectrogram representations, while Recurrent Neural Networks (RNNs) and their variants capture temporal dependencies inherent in audio data. Spectrogram-based feature extraction plays a crucial role in representing audio signals in a time-frequency domain, allowing the models to learn subtle distinc-

tions between sound classes.

By fusing the strengths of machine learning and deep learning techniques, this project aims to build a robust system capable of accurately classifying military audio signals. The outcome offers significant potential for enhancing real-time surveillance and threat detection in complex operational environments.

2. Dataset

Initially, we were provided with three CSV files: mad-dataset-annotation.csv, test.csv, and training.csv. Afterward, we downloaded the relevant military-related audio files from YouTube using links from the dataset provided in the mad-data-annotation.csv file. From that, 34 failed to download due to unavailability, which resulted in missing audio files. To process the audio data, we matched and extracted the correct audio segments based on the columns video-num, file-id, Start-time, and End-time from the training.CSV files. These columns served as references for identifying the specific audio clips associated with each entry. After segmenting the videos, we obtained 7,837 samples instead of the expected 8,086 due to missing videos. Next, we filtered the training and testing datasets to include only the rows where the video-num corresponds to the available.wav files in the specified directory. The filtered datasets were then saved as filtered-training.csv and filtered-testing.csv. This ensures that the datasets only contain entries with corresponding audio files.

2.1. Dataset Details

The dataset consists of two main CSV files: filtered-training.csv and filtered-testing.csv, which store information about audio files related to military sounds. These files contain columns with details such as the audio file paths, corresponding labels, YouTube video titles, URLs, and the start and end times of the audio segments. Each entry in the CSV files is linked to a specific audio file stored in directories under data/MAD-dataset/training and data/MAD-dataset/test.

filtered-training.csv contains the training data, with audio files stored in the training subdirectory. filtered-testing.csv contains the testing data, with audio files located in the test subdirectory.

3. Data Preprocessing and Feature Extraction

3.1. MFCC Feature extraction

We obtained MFCC features and summarized them with mean, variance, minimum, and maximum statistics. These characteristics were saved along with their labels in a CSV file and converted into vectors for uniform input to models. MFCCs capture frequency-domain characteristics aligned with human auditory perception, effectively encoding spectral and temporal information crucial for audio classification.

3.2. Mel Spectrogram

The audio data transformed into Mel spectrograms with a sampling rate of 22,500 Hz and 128 Mel bins. Each spectrogram was resized to fixed width of 216 to ensure uniform dimensions. These spectrograms were stored in NumPy arrays (`X.npy` and `y.npy` for training, `X_test.npy` and `y_test.npy` for testing) along with their labels for efficient use in machine learning pipelines.

3.3. Class Imbalances

On observation there was slightly class imbalances around 40-50. So this will not effect that much while training.

3.4. Handling Null values

There were 269 null values in the Youtube URL column of the training set and 78 null values in the test set. Given the total of around 6000 training and 1000 testing samples, we decided to drop the rows with null values as they did not significantly impact the analysis.

3.5. Scaling and Label Encoding

In the data preprocessing phase, the features (`X_train` and `X_test`) were standardized using `StandardScaler` to ensure a mean of zero and a standard deviation of one. The target labels (`y_train` and `y_test`) were encoded using `LabelEncoder` to convert categorical labels into numerical form, and the encoded labels were then converted into one-hot vectors using `to_categorical` for multi-class classification.

4. Exploratory Data Analysis (EDA)

In exploratory data Analysis we have seen the class distributions, and the average duration for each class as mentioned below.

Computed the average, minimum, and maximum lengths for each label in the dataset. Maintained consistency in sampling rates and lengths throughout all audio files in both the training and testing datasets.

Spectrograms were generated from each class using Short-Time Fourier Transform to visualize and analyze

Label	Count
1	1143
4	883
5	871
3	823
6	806
2	732
0	712

Table 1. Class Distribution for train set

Label	Count
1	262
0	202
6	129
4	108
2	98
3	84
5	76

Table 2. Class Distribution for test set

Label	Average Duration
0	5.543056
1	4.762232
2	5.086066
3	4.007326
4	5.715742
5	6.740528
6	6.368098

Table 3. Average Duration Per Class

the frequency patterns of different military sounds, aiding in their classification.

5. Model Training and Methodology

5.1. Training on MFCC features

- ML models: Each models –Random Forest, Logistic Regression, Support Vector Machine (RBF Kernel), K-Nearest Neighbors, XGBoost, and MLP– were trained on MFCC features.
- DL models: ANN, CNN, RNN were trained on MFCC features. The ANN had three dense layers (128, 64, num_classes) with dropout (0.3) and softmax output. The CNN used Conv1D (32, 64 filters, kernel size 3), MaxPooling, flatten, dense (64), dropout (0.3), and softmax. The RNN included LSTM (64 units), dense (64), dropout (0.3), and softmax. All models used the Adam optimizer, categorical crossentropy loss, 30 epochs, and batch size 32, with reshaped inputs for CNN and RNN.

5.2. Training on Mel Spectrogram Features

- Machine Learning Models: Machine learning model such as random forest has used Classifier, using 100 decision trees. It preprocesses the data by normalizing and flattening it.

- **Deep Learning Models:** Various deep learning models, such as Convolutional Neural Networks (CNNs) and Artificial Neural Networks (ANNs), were explored to effectively classify different types of military audio signals, including gunshots, explosions, and speech commands. The number of parameters in the CNN and ANN models varied significantly, with a total range from 110,409 to 148,167 parameters. This range reflects the complexity and architecture variations across different models used during training.

6. Results and Comparison Models

6.1. Trained results on MFCC

Model	Accuracy	Precision	Recall	F1
Random Forest	89.79	89.81	89.79	89.79
Logistic	67.71	67.74	67.71	67.61
SVM	76.28	76.66	76.28	76.37
K-Nearest Neighbors	70.15	71.02	70.15	70.25
XGBoost	89.98	90.00	89.98	89.97
MLP	88.83	88.84	88.83	88.82
ANN	80.23	80.55	80.23	80.29
CNN	88.86	89.00	88.86	88.83
RNN	55.97	55.00	56.86	55.83

Table 4. Performance Metrics of Various Models

The evaluation of different models indicates a slight overfitting in more complex models like Random Forest, XGBoost, and MLP, which demonstrate flawless training accuracy (approximately 99.85%) yet significant reductions in test accuracy (90.80%, 90.98%, and 89.84%, respectively), implying they retain the training data and have difficulty generalizing effectively. Conversely, less complex models such as Logistic Regression and SVM exhibit more consistent performance, achieving accuracies of approximately 70% and 79% for both training and testing, suggesting superior generalization despite being less complex. K-Nearest Neighbors exhibits some overfitting, demonstrated by a moderate decrease from training (73.72%) to testing (70.15%). The ANN, CNN, and RNN models show comparable patterns, with differences noted between training and testing accuracy. To minimize overfitting, to overcome the overfitting we can use regularization methods, cross-validation, and model simplification, particularly for tree-based and neural network architectures.

6.2. Trained results on Mel Spectrogram

Model	Accuracy	Precision	Recall	F1
Random Forest	0.4545	0.4425	0.4505	0.4608
ANN	0.3557	0.2526	0.3356	0.2576
CNN	0.6307	0.6227	0.6501	0.6292

Table 5. Error Metrics on Test Set

At first, the model showed overfitting in CNN, reaching 81% accuracy on the training set while only obtaining 60% on the testing set. To tackle this, we added

one or two dropout layers, which greatly enhanced the model's performance by mitigating overfitting. Furthermore, the accuracy was lacking when the labels were not encoded in a one-hot format. Following the implementation of one-hot encoding, the accuracy increased even more, and the model showed enhanced generalization without indications of overfitting. The model starts to converge reliably after 23–31 epochs.

7. Conclusion

This research explores the use of machine learning and deep learning methods for categorizing military audio signals. Employing Mel spectrogram and MFCC features, multiple models were created and evaluated. MFCC features analyzed by a deep learning model reached the highest accuracy of 89%, showcasing their effectiveness. Machine learning models employing MFCC features attained 56% accuracy. In the meantime, Mel spectrogram features demonstrated encouraging outcomes, reaching 56% accuracy with CNNs but around 45% with Random Forest, highlighting their applicability for deep learning uses. These results emphasize the combined advantages of each feature extraction method and establish a foundation for improving audio surveillance systems for military applications. MFCC features showed better performance.

8. GitHub Repository

[Implmentation of ML and DL Models](#)

9. References

- [1] [A military Audio Dataset for Situational Awareness and Surveillance](#)
- [2] [Github Military Adio Dataset](#)
- [3] [scikit-learn classes and various functions documentation](#)
- [4] [Matplotlib API's Documentation](#)
- [4] [Feature Extraction MFCC](#)
- [5] [Feature Extraction Mel Spectrogram](#)